
0.1 Definitions

The agent is the main entity making the decisions within the environment and responding to feedback from the environment. Each action the agent does nets a reward. High reward might mean a good action in the short run; however, going down that path may not yield a higher total reward. Thus, we introduce the following notion to fix this issue, otherwise the algorithm will act greedily:

Definition 0.1 (Policies/Values). *A policy is a function that takes the current state and decides the next immediate action. A value function predicts the expected total reward after an action is taken. RL models look to improve the value obtained. Policy gradient methods implement only policy functions. Some RL systems implement value functions and implicitly derive the policy function, while some have both as separate processes.*

We also introduce the following two concepts. Branching off the idea of a configuration space, we introduce the notion of a state space. In our explorations of robots and their configuration spaces, we considered the system to be the robot itself (with no external environment).

In the case of reinforcement learning, in order for the agent to receive a reward, there must be an external environment, which itself has a configuration.

Definition 0.2. *The state space S is defined as a manifold with local coordinates from the configuration space of the environment plus agent and velocity values if applicable.*

In a multi-armed bandit problem (or n -armed bandit), we have multiple choices each with their unknown probability distributions. The agent must choose based on what maximizes the reward and must learn the probability distributions of each (this is the dilemma of exploration v.s. exploitation). This situation has only one state because the choices and their distributions remain the same no matter the action of the agent. Each time step, the agent has the same n choices.

Similarly, we define:

Definition 0.3. *The action space A is the collection of all possible actions the agent can execute within the environment.*

In the case of the n -bandit problem, the action space is discrete and has n actions. Now in order to decide what action to take next, we require a policy. We show how in this one state system, we can derive an implicit policy from an "action-value" function. The action value function is a map:

$$Q : S \times A \rightarrow \mathbb{R}$$

Given some $a \in A$, let $q(s, a)$ be the true mean value of the action a in the state s . Since the n -bandit problem considers one state, we can write this as $Q(a)$, which is fixed throughout.

Then, we can define the sample average value through the following notions. Let t be the enumeration variable for discrete time steps and $\{a_t\}_{t \in \mathbb{N}} \in A^{\mathbb{N}}$ be the sequence of past actions (with time steps not attained marked with an empty action):

1. Let $N_{t_0}(a) \in \mathbb{N}$ be the number of times a has been chosen for $t \leq t_0$
2. Let R_i be the i -th reward for the action a where $i \in [0, N_{t_0}(a)] \cap \mathbb{N}$

Then the estimated true value is:

$$Q_{t_0}(a) := \frac{\sum_{i=0}^{N_{t_0}(a)} R_i(a)}{N_{t_0}(a)}$$

This gives the running average reward of an action a up to the t_0 time step. By the law of large numbers, we have that:

$$\lim_{t \rightarrow \infty} Q_t(a) = Q(a)$$

We can derive a greedy implicit policy in this case, which simply gives us the action that provides the highest estimated true value at the current time step t_0 :

$$a_{t_0+1} := \arg \max_{a \in A} Q_{t_0}(a)$$

In ideality, we want the algorithm for all $a \in A$ to narrow down on all n true values $Q(a)$, so that the agent ends up eventually consistently selecting the action with the highest true reward, which we call the optimal action.

The issue is that the greedy algorithm repeatedly chooses the highest predicted value, so it can get stuck choosing a suboptimal action till the reward function corrects the estimated value, pushing the greedy agent to choose other actions.

However, it can get stuck on suboptimal actions altogether. Consider a 2-bandit problem with the optimal action a_0 and the suboptimal action a_1 . Lets define the true values:

$$Q(a_0) = 15 \wedge Q(a_1) = 10$$

Now assume the agent completes two time steps in which it explores both actions, with $Q_2(a_0) = 5$ and $Q_2(a_1) = 15$. The greedy agent will then choose a_1 as the next action.

The environment will attempt to correct this behavior by lowering the reward and by law of large numbers will push $Q_t(a_1) \rightarrow 10$. The greedy algorithm will still think it is the optimal action as $Q_2(a_0) = 5 < 10$, so it will never call upon a_0 and will be stuck in a suboptimal loop.

Now consider the following formula manipulation:

$$\begin{aligned} Q_t(a) &:= \frac{\sum_{i=0}^{N_t(a)} R_i(a)}{N_t(a)} = \frac{1}{N_t(a)} \left(R_{N_t(a)}(a) + \sum_{i=0}^{N_t(a)-1} R_i(a) \right) \\ &= \frac{1}{N_t(a)} (R_{N_t(a)}(a) + (N_t(a) - 1)Q_{t'}(a) + Q_{t'}(a) - Q_{t'}(a)) \end{aligned}$$

where t' is the time where $N_{t'}(a) = N_t(a) - 1$. Then:

$$= \frac{1}{N_t(a)} (R_{N_t(a)}(a) + N_t(a)Q_{t'}(a) - Q_{t'}(a)) = Q_{t'}(a) + \frac{1}{N_t(a)} (R_{N_t(a)}(a) - Q_{t'}(a))$$

so it suffices to store $Q_{t'}(a)$ and $N_{t'}(a)$, which can be used to update the estimated expected value the next time the action is taken under the current policy.